

Interactive / complex / 10

IC 1
IC 2
IC 3
IC 4
IC 5
IC 6
IC 7
IC 8
IC 9
IC 10
IC 11
IC 12
IC 13
IC 14v1
IC 14v2

query	Interactive / complex / 10			
title	Friend recommendation			
pattern	The diagram illustrates the query pattern. It starts with a <code>person: Person</code> box with a variable <code>id = \$personId</code>. This box is connected via a <code>knows*2..2</code> relationship to a <code>foaf: Person</code> box. The <code>foaf: Person</code> box has a filter: <code>(month(birthday) = \$month and day(birthday) ≥ 21) or (month(birthday) = \$month+1 and day(birthday) < 22)</code>. Below this box are variables <code>id</code>, <code>firstName</code>, <code>lastName</code>, and <code>gender</code>. The <code>foaf: Person</code> box is connected via an <code>isLocatedIn</code> relationship to a <code>city: City</code> box, which has a variable <code>name</code>. Below the main pattern are two sub-patterns: common: Shows <code>person: Person</code> connected to <code>Tag</code> via <code>hasInterest</code>, and <code>foaf: Person</code> connected to <code>Post</code> via <code>hasCreator</code>. <code>Tag</code> is connected to <code>Post</code> via <code>hasTag</code>. A <code>count</code> box is shown between <code>Post</code> and <code>foaf: Person</code>. uncommon: Similar to the common pattern, but the <code>hasInterest</code> relationship is negated (indicated by a red arrow and <code>«neg»</code>), and the <code>count</code> box is also present.			
description	Given a start Person with ID <code>\$personId</code>, find that Person’s friends of friends (<code>foaf</code>) – excluding the start Person and his/her immediate friends –, who were born on or after the 21st of a given <code>\$month</code> (in any year) and before the 22nd of the following month. Calculate the similarity between each friend and the start person, where <code>commonInterestScore</code> is defined as follows: • <code>common</code> = number of Posts created by friend, such that the Post has a Tag that the start person is interested in • <code>uncommon</code> = number of Posts created by friend, such that the Post has no Tag that the start person is interested in • <code>commonInterestScore</code> = <code>common</code> - <code>uncommon</code>			
params	1	<code>\$personId</code>	ID	
	2	<code>\$month</code>	32-bit Integer	Between 1 and 12. Implementations may also pass the next month as an additional <code>\$nextMonth</code> parameter
result	1	<code>foaf.id</code>	ID	R
	2	<code>foaf.firstName</code>	String	R
	3	<code>foaf.lastName</code>	String	R
	4	<code>commonInterestScore</code>	32-bit Integer	A
	5	<code>foaf.gender</code>	String	R
	6	<code>city.name</code>	String	R
sort	1	<code>commonInterestScore</code>	↓	
	2	<code>foaf.id</code>	↑	
limit	10			
CPs	2.3, 3.3, 4.1, 4.2, 5.1, 5.2, 6.1, 7.1, 8.6			
relevance	This query looks for paths of length two, starting from a Person and ending at the friends of their friends. It does widely scattered graph traversal, and one expects no locality of in friends of friends, as these have been acquired over a long time and have widely scattered identifiers. The join order is simple but one must see that the anti-join for “not in my friends” is better with hash. Also the last pattern in the scalar sub-queries joining or anti-joining the Tags of the candidate’s Posts to interests of self should be by hash.			